

TRƯỜNG ĐẠI HỌC THĂNG LONG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

ĐỀ TÀI: ỨNG DỤNG QUẢN LÝ KHO SIÊU THỊ

Giảng viên hướng dẫn:

Trần Văn Nam

Thành viên nhóm 3:

A52347 – Nguyễn Việt Cường

A52455 – N.Xuân Thành Trung

A52542 – Hoàng Gia Huy

Lớp

TI3801

TI3801

TI3801

HÀ NỘI – 2026

Lời cảm ơn

Nhóm 6 chúng em xin gửi lời cảm ơn chân thành đến các thầy cô khoa Công nghệ Thông tin - Trường Đại học Thăng Long đã tận tình giảng dạy và cung cấp nền tảng kiến thức vững chắc về lập trình C++ và cấu trúc dữ liệu. Những kiến thức này là tiền đề quan trọng giúp chúng em hoàn thiện bài tập này. Xin trân trọng cảm ơn!

Lời cam đoan

Nhóm 6 xin cam đoan đây là công trình nghiên cứu và phát triển của riêng nhóm. Các đoạn code, thuật toán, phân tích hệ thống được tự xây dựng dựa trên kiến thức đã học. Các tài liệu tham khảo được trích dẫn rõ ràng. Nếu có sự sao chép bất hợp pháp, Nhóm xin chịu hoàn toàn trách nhiệm trước giảng viên.

Mục lục

1. GIỚI THIỆU

Chương trình này được xây dựng nhằm mô phỏng việc quản lý kho siêu thị qua hệ thống, cho phép:

- A. Nhập hàng vào kho
- B. Xuất hàng ra khỏi kho
- C. Kiểm tra các mặt hàng sắp hết hạn
- D. Kiểm tra các mặt hàng sắp hết hàng
- E. Thống kê tồn kho
- F. Báo cáo thông minh
- G. Tối ưu kho
- H. Lưu và lấy dữ liệu quản lý kho thông qua file .txt

Chương trình sử dụng ngôn ngữ C++, áp dụng cấu trúc dữ liệu public struct, mảng động và xử lý file.

2. CƠ SỞ LÝ THUYẾT

1. Quản lý Nghiệp vụ Nhập - Xuất

+ **Nhập hàng:** Thực hiện ghi nhận thông tin hàng hóa mới, cập nhật số lượng tồn kho và lưu trữ thông tin về các nhà cung cấp

+ **Xuất hàng:** Xử lý các đơn hàng bán ra, tự động khấu trừ số lượng tồn kho thực tế và tạo hóa đơn xuất hàng cho khách.

2. Hệ thống Cảnh báo và Thống kê

+ **Cảnh báo hết hạn:** Tự động so sánh ngày hiện hành với hạn sử dụng (HSD) của sản phẩm để đưa ra thông báo nhắc nhở trước 7 ngày.

+ **Cảnh báo hết hàng:** Phát tín hiệu thông báo khi số lượng sản phẩm trong kho thấp hơn mức tồn tối thiểu đã được cài đặt trước đó.

+ **Thống kê tồn kho:** Tính toán tổng giá trị tài sản hiện có trong kho theo công thức:

$$\text{Tổng giá trị} = \sum (\text{Số lượng} \times \text{Giá nhập})$$

3. Các tính năng đặc biệt (Nâng cao)

+ **Báo cáo thông minh:** Phân tích dữ liệu và xu hướng bán hàng theo các mốc thời gian (ngày, tuần, tháng) để dự báo nhu cầu nhập hàng.

+ **Quản lý Nhà cung cấp (NCC):** Lưu trữ hồ sơ đối tác, thực hiện đánh giá chất lượng và độ tin cậy của các nhà cung cấp.

+ **Tối ưu kho:** Đề xuất các vị trí lưu trữ hàng hóa hợp lý dựa trên tần suất xuất hàng (ưu tiên vị trí thuận tiện cho các mặt hàng bán chạy).

4. Yêu cầu kỹ thuật chung

+ **Chức năng CRUD đầy đủ:** Đảm bảo hệ thống có thể Thêm (Add), Sửa (Edit), Xóa (Delete), Tìm kiếm (Search) và Sắp xếp (Sort) dữ liệu linh hoạt.

+ **Lưu trữ dữ liệu bền vững:** Toàn bộ dữ liệu phải được ghi xuống tệp tin (.txt hoặc binary) để đảm bảo không bị mất dữ liệu sau khi tắt chương trình.

+ **Xử lý lỗi và Ngoại lệ:** Sử dụng cơ chế try-catch để kiểm soát các tình huống như lỗi mở file, cấp phát bộ nhớ thất bại hoặc nhập sai kiểu dữ liệu, giúp ứng dụng vận hành ổn định.

+ **Giao diện và AI:** Xây dựng menu điều hướng logic trên Console và ứng dụng AI để tối ưu hóa cũng như chú thích mã nguồn chuyên nghiệp, và có thể trình bày lại.

3. CẤU TRÚC DỮ LIỆU

3.1. Thư viện

- **<iostream>:** Cung cấp các đối tượng để nhập và xuất dữ liệu cơ bản
- **<string>:** Quản lý và thao tác với các văn bản dài.

- **<iomanip>**: Cung cấp các công cụ để định dạng cách hiển thị dữ liệu đầu ra.
- **<ctime>**: Cung cấp các hàm và kiểu dữ liệu liên quan đến ngày giờ hệ thống.
- **<fstream>**: Cho phép chương trình đọc (Read) và ghi (Write) dữ liệu vào các tệp tin trên ổ cứng.

3.2. Cấu trúc struct

3.2.1. Cấu trúc ngày (Ngay)

```
struct Ngay{
    int ngay, thang, nam;
};
```

Thiết lập hàm struct Ngay và tạo ra các biến ngay, thang, nam, lưu trữ ngày, tháng, năm. Struct này được sử dụng trong struct MatHang và phần lớn các hàm công dụng còn lại.

3.2.2. Cấu trúc mặt hàng (MatHang)

```
//Thông tin mặt hàng trong kho
struct MatHang {
    string maHang;
    string tenHang;
    string donViTinh;
    double giaNhap;
    double giaBan;
    int soLuongTon;
    Ngay hanSuDung;
    int mucTonToiThieu;
};
```

Thiết lập hàm struct MatHang và tạo ra các biến:

maHang: Mã hàng dùng để định danh các loại sản phẩm.

tenHang: Tên đầy đủ của mặt hàng.

donViTinh: Đơn vị đo lường (Thùng, Kg,...) để chuẩn hoá số lượng

giaNhap, giaBan: Cơ sở để tính toán tổng giá trị kho hàng và doanh thu thực tế.

soLuongTon: Số lượng thực tế hiện có trong kho.

hanSuDung: Sử dụng cấu trúc lồng Ngay (Nested Struct) để quản lý thời hạn của sản phẩm.

mucTonToiThieu: Ngưỡng an toàn để hệ thống đưa ra cảnh báo nhập hàng.

Hàm này có vai trò chung tâm, lưu trữ tất cả thông tin của từng sản phẩm trong kho hàng.

3.2.3. Cấu trúc nhà cung cấp (NhaCungCap)

```
// Thông tin nhà cung cấp (dùng trong hoá đơn nhập hàng)
struct NhaCungCap {
    string tenNCC;
    string soDienThoai;
    string diaChi;
    int diemChatLuong;           // Đánh giá từ 1-5 sao, ÉP CỨNG DÙ FILE ĐÃ SAVE TRONG TXT
    int diemUyTin;              // Đánh giá từ 1-5 sao, ÉP CỨNG DÙ FILE ĐÃ SAVE TRONG TXT
};
```

Thiết lập hàm struct NhaCungCap và tạo ra các biến:

tenNCC: Tên pháp nhân của nhà cung cấp.

soDienThoai, diaChi: Thông tin liên hệ cơ bản để phục vụ việc giao dịch và vận chuyển.

diemChatLuong: Chỉ số đánh giá chất lượng sản phẩm (thang điểm 1-5).

diemUyTin: Chỉ số đánh giá mức độ tin cậy (thang điểm 1-5).

Hàm này lưu trữ, quản lý thông tin nhà cung cấp, bao gồm việc đánh giá chất lượng và độ uy tín thông qua thang điểm từ 1-5. Những thang điểm này sẽ bị ép cứng dạng số nguyên (int), thấp nhất là 1 và cao nhất là 5 khi đưa lên output hoặc khi input.

3.2.4. Cấu trúc chi tiết hoá đơn (ChiTietHoaDon)

```
//Chi tiết hoá đơn, dùng cho cả nhập và xuất hàng, lưu thông tin về mặt hàng nào, số lượng bao nhiêu, giá cả ra sao (giá nhập cho hoá đơn nhập, giá bán cho hoá đơn xuất)
struct ChiTietHoaDon {
    string maHang;
    int soLuong;
    double donGia;
};
```

Thiết lập hàm struct ChiTietHoaDon và tạo ra các biến:

maHang: Mã hàng dùng để định danh các loại sản phẩm. (Được xuất ra hoá đơn)

soLuong: Số lượng cụ thể của mặt hàng đó trong riêng hóa đơn này.

donGia: Mức giá áp dụng tại thời điểm giao dịch (Giá nhập đối với hóa đơn nhập, giá bán đối với hóa đơn xuất).

Hàm này lưu trữ thông tin của từng mặt hàng xuất hiện trên hoá đơn.

3.2.5. Cấu trúc hoá đơn nhập (HoaDonNhap)

```
// Hoá đơn nhập hàng từ nhà cung cấp
struct HoaDonNhap {
    string maHDN;
    Ngay ngayNhap;
    NhaCungCap ncc;

    ChiTietHoaDon* danhSachNhap;
    int soLuongMatHang;
    double tongTien;
};
```

Thiết lập hàm struct HoaDonNhap và tạo ra các biến:

maHDN: Mã số định danh duy nhất của hóa đơn nhập hàng.

ngayNhap: Thời điểm hàng được nhập về kho, sử dụng cấu trúc Ngay lồng bên trong.

ncc: Thông tin về nhà cung cấp lô hàng này, sử dụng cấu trúc NhaCungCap.

danhSachNhap: Con trỏ kiểu ChiTietHoaDon, dùng để quản lý một mảng động các mặt hàng có trong hóa đơn.

soLuongMatHang: Lưu số lượng loại mặt hàng thực tế trong hóa đơn, đồng thời là kích thước của mảng động danhSachNhap.

tongTien: Tổng giá trị tài chính của toàn bộ hóa đơn nhập hàng.

Hàm này lưu trữ, quản lý toàn diện các lần nhập hàng từ phía nhà cung cấp.

3.2.6. Cấu trúc hoá đơn xuất (HoaDonXuat)

```
// Hoá đơn xuất hàng đến khách hàng
struct HoaDonXuat {
    string maHDX;
    Ngay ngayXuat;
    string kháchHang;

    ChiTietHoaDon* danhSachBan;
    int soLuongMatHang;
    double tongTien;
};
```

Thiết lập hàm struct HoaDonNhap và tạo ra các biến:

maHDX: Mã định danh duy nhất của hóa đơn xuất hàng.

ngayXuat: Ngày thực hiện giao dịch xuất kho, sử dụng struct Ngay lồng bên trong.

kháchHang: Tên cá nhân hoặc tổ chức mua hàng.

danhSachBan: Con trỏ quản lý mảng động các phần tử ChiTietHoaDon.

soLuongMatHang: Số lượng loại sản phẩm có trong hóa đơn (đồng thời là kích thước

mảng động).

tongTien: Tổng giá trị thanh toán của khách hàng.

Hàm này lưu trữ, quản lý toàn diện các lần xuất mặt hàng cho khách hàng.

3.2.7. Cấu trúc doanh số mặt hàng (DoanhSoMatHang)

```
// Cấu trúc hỗ trợ báo cáo thông minh: theo dõi doanh số bán hàng từng sản phẩm
struct DoanhSoMatHang {
    string maHang;
    int soLuongBan;
};
```

Thiết lập hàm struct HoaDonNhap và tạo ra các biến:

maHang: Mã định danh của sản phẩm cần thống kê. (cho báo cáo thông minh)

soLuongBan: Tổng số lượng đã bán ra của mặt hàng đó tính trên toàn bộ lịch sử giao dịch.

Hàm này tổng hợp dữ liệu, chủ yếu phục vụ cho việc báo cáo thông minh.

4. CÁC CHỨC NĂNG CHÍNH

4.1. Nhập hàng

```
// Thêm sản phẩm mới vào kho, tự động mở rộng nếu cần
void themMatHang(MatHang*& kho, int& nKho, int& sucChua, string newMaHang, Ngay newHsd){
    // Mở rộng dung lượng nếu đầy (thế tăng 2x+1 giảm lượng cấp phát)
    if (nKho >= sucChua){
        int newSucChua = sucChua * 2 + 1;
        // Tóm gọn thuật toán như sau: * 2 để giảm lượng làm việc của máy, do mỗi lần tạo và xoá array sau mỗi lần đầy kho rất tốn cpu,
        // +1 để đảm bảo trường hợp sucChua = 0 (0 * 2 + 1 = 0)

        MatHang* temp = new MatHang[newSucChua];
        for (int i = 0; i < nKho; i++) temp[i] = kho[i];
        delete[] kho;
        kho = temp;
        sucChua = newSucChua;
    }

    // Nhập thông tin sản phẩm
    kho[nKho].maHang = newMaHang;
    cout << "-Nhập tên mặt hàng: ";
    cin.ignore(); getline(cin, kho[nKho].tenHang);
    cout << "-Nhập đơn vị tính mặt hàng: ";
    getline(cin, kho[nKho].donViTinh);
    cout << "-Nhập giá nhập mặt hàng: "; cin >> kho[nKho].giaNhap;
    cout << "-Nhập giá bán mặt hàng: "; cin >> kho[nKho].giaBan;
    kho[nKho].hanSuDung = newHsd;
    cout << "-Nhập mức tồn tối thiểu: "; cin >> kho[nKho].mucTonToiThieu;

    kho[nKho].soLuongTon = 0;

    nKho++;
}
```

kho: con trỏ quản lý mảng động danh sách mặt hàng.

nKho: số lượng mặt hàng hiện tại.

sucChua: kích thước tối đa của mảng hiện tại.

B1: Sử dụng lệnh “if (nKho >= sucChua)” để kiểm tra tràn mảng. Nếu mảng đã đầy, sẽ tự động cấp phát một mảng mới với kích thước lớn hơn theo công thức “sucChua * 2 + 1” (+1 để phòng trường hợp sucChua = 0) rồi sao chép dữ liệu cũ sang mảng temp, giải phóng bộ nhớ mảng cũ bằng “delete[]” và gán lại vào mảng chính.

B2: Sử dụng “cin.ignore()” để xóa bộ nhớ đệm (buffer) nhằm tránh lỗi lệnh getline bị trôi không cho người dùng nhập liệu.

B3: Tăng biến đếm nKho++ để xác nhận đã thêm thành công.

4.2. Xử lý xuất hàng

4.2.1. Kiểm tra trạng thái kho hàng

```
// Xử lý xuất hàng: thu thập thông tin khách hàng và chỉ tiết sản phẩm bán
void xuLyXuatHang(MatHang*& kho, int& nKho, int& sucChuaKho, HoaDonXuat*& dsHDX, int& nHDX, int& sucChuaHDX){
    // Kiểm tra xem kho có hàng không
    if(nKho == 0){
        cout << "Xin loi, kho dang trong, hien khong co mat hang nao de ban.\n";
        return;
    }
}
```

Dòng “if (nKho == 0)” được sử dụng để kiểm tra kho, nếu rỗng thì lập tức return thoát hàm để tránh lỗi logic xuất hàng ảo.

4.2.2. Khởi tạo và thu thập thông tin hóa đơn xuất

```
HoaDonXuat hdx;

cout << "Nhap ma hoa don xuat: "; cin >> hdx.maHDX;
cout << "Nhap ten khach hang: ";
cin.ignore(); getline(cin, hdx.khachHang);
cout << "Nhap ngay xuat hang (ngay -> thang -> nam): "; cin >> hdx.ngayXuat.ngay >> hdx.ngayXuat.thang >> hdx.ngayXuat.nam;

cout << "Chon so luong mat hang muon mua: "; cin >> hdx.soLuongMatHang;

// Cấp phát mảng động cho chỉ tiết hàng trong hoá đơn:
hdx.danhSachBan = new ChiTietHoaDon[hdx.soLuongMatHang];
hdx.tongTien = 0;
```

B1: Tạo một biến cục bộ hdx có kiểu HoaDonXuat để lưu tạm các thông tin của phiên giao dịch.

B2: Sử dụng “cin.ignore()” để xóa vùng nhớ đệm trước khi dùng getline nhập tên khách hàng có khoảng trắng.

B3: Áp dụng kỹ thuật cấp phát mảng động “new ChiTietHoaDon[]” để tạo ra một danh sách chứa đúng số lượng loại hàng khách chọn, giúp tối ưu RAM. Tổng tiền hóa

đơn được khởi tạo bằng 0.

4.2.3. Quét và lọc các lô hàng tồn kho theo mã

```
for (int i = 0; i < hdx.soLuongMatHang; i++){
    string maCanTim;
    cout << "Nhap ma hang thu " << i+1 << ":"; cin >> maCanTim;

    // Đếm số lô hàng có sẵn cho sản phẩm này (các lô khác hạn sử dụng)
    int soLoThucTe = 0;
    for (int j = 0; j < nKho; j++) {
        if (kho[j].maHang == maCanTim && kho[j].soLuongTon > 0) soLoThucTe++;
    }

    if (soLoThucTe == 0) {
        cout << "Loi: Het hang.\n"; i--; continue;
    }

    // Lưu vị trí của các lô hàng có sẵn
    int* dsViTriDong = new int[soLoThucTe];
    int chiSo = 0;
    for (int j = 0; j < nKho; j++) {
        if (kho[j].maHang == maCanTim && kho[j].soLuongTon > 0) dsViTriDong[chiSo++] = j;
    }
}
```

Đây là phần xử lý vấn đề một mặt hàng có nhiều hạn sử dụng khác nhau theo từng lô hàng.

B1: Vòng lặp for đầu tiên đếm xem trong kho có bao nhiêu lô hàng khớp mã maCanTim và vẫn còn tồn kho (soLuongTon > 0).

B2: Cấp phát một mảng con trỏ phụ dsViTriDong có kích thước đúng bằng số lô vừa đếm được.

B3: Vòng lặp thứ hai chạy để lưu lại vị trí (index) thực tế của các lô hàng này trong mảng kho gốc vào mảng phụ dsViTriDong. Việc này giúp hiển thị danh sách để khách hàng có thể chọn chính xác xuất hàng từ lô nào (ví dụ ưu tiên lô cận date xuất trước).

4.2.4. Hiển thị danh sách và kiểm soát lựa chọn lô hàng

```
// Hiển thị danh sách các lô hàng để khách chọn
cout << "Tìm thấy " << soLoThucTe << " lô hàng. Chọn STT:\n";
for (int j = 0; j < soLoThucTe; j++) {
    int vt = dsViTriDong[j];
    cout << j + 1 << ". HSD: " << kho[vt].hanSuDung.ngay << "/" << kho[vt].hanSuDung.thang << " - Ton: " << kho[vt].soLuongTon << " (" << kho[vt].donViTinh << ")\n";
}

int choice;
do {
    cout << "Chọn STT lô hàng muốn xuất (1-" << soLoThucTe << "): ";
    cin >> choice;

    if (choice < 1 || choice > soLoThucTe) {
        cout << "STT không hợp lệ, mời bạn nhập lại!\n";
    }
} while (choice < 1 || choice > soLoThucTe);
```

Vòng lặp for duyệt qua mảng phụ dsViTriDong để lấy ra vị trí (vt) của các lô hàng hợp lệ. Sau đó, hệ thống truy xuất vào mảng kho gốc để in ra màn hình Số thứ tự (STT), Hạn sử dụng (HSD) và Số lượng tồn kho của từng lô, giúp khách hàng có thông tin trực quan để đưa ra quyết định.

Vòng lặp do-while ép người dùng phải chọn một STT hợp lệ. Cấu trúc “if (choice < 1 || choice > soLoThucTe)” kiểm tra nếu số nhập vào nằm ngoài khoảng danh sách vừa hiển thị thì sẽ báo lỗi và yêu cầu nhập lại, giúp chương trình không bị lỗi truy xuất mảng (out of bounds).

4.2.5. Khấu trừ số lượng tồn kho và tính tiền

```
if (kho[vtThucTe].soLuongTon >= sl) {
    // Cập nhật tồn kho (trừ = xuất hàng)
    capNhatTonKho(kho, nkho, maCanTim, kho[vtThucTe].hanSuDung, -sl);

    hdx.danhSachBan[i] = {maCanTim, sl, kho[vtThucTe].giaBan};
    hdx.tongTien += (sl * hdx.danhSachBan[i].donGia);
    cout << "Xác nhận bán " << sl << " " << kho[vtThucTe].donViTinh << ".\n";
}
else {
    cout << "Lỗi: Không đủ hàng.\n"; i--;
}

delete[] dsViTriDong;
```

Sau khi người dùng chọn lô hàng (choice), hệ thống quy đổi ngược lại vị trí thực trong kho qua biến vtThucTe.

B1: Thuật toán kiểm tra an toàn “if (kho[vtThucTe].soLuongTon >= sl)” đảm bảo khách không thể mua số hàng đang có.

B2: Hàm capNhatTonKho được tái sử dụng từ phần nhập hàng nhưng truyền vào giá trị âm (-sl) để trừ đi số lượng đã bán.

B3: Cập nhật chi tiết vào mảng danhSachBan, cộng dồn số tiền vào tongTien và cuối cùng là giải phóng bộ nhớ của mảng phụ dsViTriDong để tránh rò rỉ bộ nhớ (memory leak).

4.2.6. Lưu hóa đơn xuất

```
if(nHDX >= sucChuaHDX){
    int newSucChua = sucChuaHDX * 2 + 1;
    HoaDonXuat* temp = new HoaDonXuat[newSucChua];
    for (int i = 0; i < nHDX; i++) temp[i] = dsHDX[i];
    delete[] dsHDX;
    dsHDX = temp;
    sucChuaHDX = newSucChua;
}

dsHDX[nHDX++] = hdx;
cout << "-Xu ly xuat hang thanh cong. Tong tien: " << hdx.tongTien << ".\n";
```

Giai đoạn cuối cùng tương tự như chức năng thêm mới mặt hàng. Thuật toán kiểm tra sức chứa của mảng động dsHDX, tự động mở rộng gấp đôi nếu đầy. Cuối cùng, gán toàn bộ thông tin hóa đơn hdx vừa tạo vào mảng tổng và tăng biến đếm nHDX++.

4.3. Cảnh báo hết hạn

4.4. Cảnh báo hết hàng

```
void thongKeTonKho(const MatHang* Kho, int nKho) {
    cout << "\n\n";
    double tongGiaTri = 0;

    for (int i = 0; i < nKho; i++) {
        double giaTri = kho[i].soluongTon * kho[i].giaNhap;
        tongGiaTri += giaTri;

        cout << "Ma: " << kho[i].maHang << " | Ten: " << kho[i].tenHang << " | SL: " << kho[i].soluongTon << " * " << kho[i].donViTinh << " | Tong gia tri nhap: " << fixed << setprecision(0) << giaTri << " VND\n";
    }

    cout << "TONG GIA TRI KHO: " << fixed << setprecision(0) << tongGiaTri << "\n";
    cout << defaultfloat;
}
```

B1: Khởi tạo biến “bool coHangThieu = false” để theo dõi xem có bất kỳ mặt hàng nào trong kho bị thiếu hụt so với quy định hay không.

B2: Vòng lặp for thực hiện duyệt qua toàn bộ các mặt hàng đang lưu trữ trong mảng kho (từ vị trí 0 đến nKho - 1).

B3: Tại mỗi mặt hàng, thuật toán sử dụng câu lệnh if để kiểm tra xem số lượng thực

tế (kho[i].soLuongTon) có bị tụt xuống dưới định mức an toàn đã cài đặt (kho[i].mucTonToiThieu) hay không.

B4: Nếu điều kiện trên là đúng (số lượng tồn nhỏ hơn mức tối thiểu), chương trình sẽ lập tức in ra thông tin cảnh báo chi tiết bao gồm: Mã hàng, Tên hàng, Số lượng tồn kho hiện tại và Mức tối thiểu yêu cầu. Đồng thời, coHangThieu được chuyển sang trạng thái true

B5: Sau khi vòng lặp kết thúc việc quét toàn bộ kho, coHangThieu vẫn giữ nguyên giá trị false (nghĩa là không có mặt hàng nào vi phạm điều kiện thiếu hàng), hệ thống sẽ in ra thông báo

4.5. Thống kê tồn kho

```
// Thống kê giá trị tồn kho theo từng sản phẩm
void thongKeTonKho(const MatHang* kho, int nKho) {
    cout << "\n\n";
    double tongGiaTri = 0;

    for (int i = 0; i < nKho; i++) {
        double giaTri = kho[i].soLuongTon * kho[i].giaNhap;
        tongGiaTri += giaTri;

        cout << "Ma: " << kho[i].maHang << " | Ten: " << kho[i].tenHang << " | SL: " << kho[i].soLuongTon << " | " << kho[i].donViTinh << " | Tong gia tri nhap: " << giaTri << " VND\n";
    }

    cout << "TONG GIA TRI KHO: " << tongGiaTri << "\n";
}
```

4.5.1. Khai báo hàm:

kho: Mảng chứa thông tin các mặt hàng trong kho (truyền hằng số const để đảm bảo dữ liệu không bị thay đổi trong quá trình thống kê).

nKho: Số lượng mặt hàng hiện có.

4.5.2. Các bước triển khai logic:

B1: Khởi tạo biến “tongGiaTri = 0” để tích lũy giá trị của toàn bộ kho.

B2: Sử dụng vòng lặp for để duyệt qua từng mặt hàng từ vị trí đầu tiên đến vị trí thứ n trong kho.

B3: Với mỗi mặt hàng, chương trình thực hiện công thức:

“giaTri = kho[i].soLuongTon * kho[i].giaNhap”

B4: Cộng dồn giá trị vừa tính được vào biến tongGiaTri.

B5: Hiển thị chi tiết từng mặt hàng bao gồm: Mã hàng, tên hàng, số lượng, đơn vị tính và tổng giá trị nhập của mặt hàng đó.

B6: Sau vòng lặp, in ra tổng giá trị kho, là con số cuối cùng sau khi đã cộng dồn toàn bộ.

4.6. Báo cáo thông minh

```
// Phân tích xu hướng bán hàng: xác định sản phẩm bán chạy nhất
void baoCaoThongMinh(const MatHang* kho, int nKho, const HoaDonXuat* dsHDX, int nHDX){
    cout << "\n === Phan tich xu huong ban hang ===\n";
    if (nHDX == 0) {
        cout << "Chua co hoa don xuất nao de thông ke.\n";
        return;
    }

    // Đếm tổng số chi tiết hàng bán
    int tongChiTiet = 0;
    for (int i = 0; i < nHDX; i++) {
        tongChiTiet += dsHDX[i].soLuongMatHang;
    }

    // Cấp phát mảng động
    string* dsMaHang = new string[tongChiTiet];
    int* dsTongSoLuong = new int[tongChiTiet];
    int soLoaiHang = 0;

    // Duyệt qua tất cả hoá đơn xuất, tính tổng số lượng bán mỗi loại hàng
    for (int i = 0; i < nHDX; i++) {
        for (int j = 0; j < dsHDX[i].soLuongMatHang; j++) {
            string maHang = dsHDX[i].danhSachBan[j].maHang;
            int soLuong = dsHDX[i].danhSachBan[j].soLuong;

            // Kiểm tra xem mã hàng này đã tồn tại chưa
            bool found = false;
            for (int k = 0; k < soLoaiHang; k++) {
                if (dsMaHang[k] == maHang) {
                    dsTongSoLuong[k] += soLuong;
                    found = true;
                    break;
                }
            }

            // Nếu chưa có thì thêm vào
            if (!found) {
                dsMaHang[soLoaiHang] = maHang;
                dsTongSoLuong[soLoaiHang] = soLuong;
                soLoaiHang++;
            }
        }
    }
}
```

4.6.1. Khai báo hàm:

kho, nKho: Thông tin danh mục hàng hóa hiện có.

dsHDX, nHDX: Danh sách toàn bộ các hóa đơn xuất hàng đã thực hiện.

Sử dụng hằng số const cho các tham số đầu vào để đảm bảo tính toàn vẹn của dữ liệu gốc, ngăn chặn các thay đổi ngoài ý muốn trong quá trình phân tích.

4.6.2. Các bước triển khai logic:

B1: Chương trình kiểm tra nếu chưa có hóa đơn xuất nào ($nHDX == 0$), hệ thống sẽ thông báo và kết thúc hàm ngay lập tức để tránh lãng phí tài nguyên xử lý.

B2: Sử dụng vòng lặp để đếm tổng số chi tiết hàng hóa (tongChiTiet) có trong tất cả

các hóa đơn. Đây là cơ sở để cấp phát bộ nhớ chính xác cho mảng phân tích tạm thời.

B3: Khởi tạo hai mảng động `dsMaHang` (lưu mã hàng duy nhất) và `dsTongSoLuong` (lưu tổng lượng bán tương ứng) với kích thước bằng `tongChiTiet`. Kỹ thuật này giúp hệ thống linh hoạt xử lý mọi quy mô giao dịch mà không bị giới hạn bởi mảng tĩnh.

B4: Duyệt qua từng hóa đơn và từng mặt hàng bên trong hóa đơn đó. Sau đó, sử dụng kỹ thuật tìm kiếm và biến cờ `found` để kiểm tra xem mã hàng hiện tại đã tồn tại trong mảng thống kê chưa. Nếu có sẽ cộng dồn số lượng bán mới vào tổng cũ, còn không sẽ thêm mã hàng mới vào mảng và ghi nhận số lượng ban đầu.

```
// Tìm mặt hàng bán chạy nhất
if (soLoaiHang > 0){
    string maHangBanChay = dsMaHang[0];
    int maxSoLuongBan = dsTongSoLuong[0];
    for (int i = 1; i < soLoaiHang; i++) {
        if (dsTongSoLuong[i] > maxSoLuongBan) {
            maxSoLuongBan = dsTongSoLuong[i];
            maHangBanChay = dsMaHang[i];
        }
    }

    cout << "Mat hang ban chay nhat: " << maHangBanChay << " | Tong so luong da ban: " << maxSoLuongBan << "\n";
    cout << "Du doan xu huong: " << maHangBanChay << " van se tiep tuc ban chay, nen du tru them mat hang nay.\n";
}

else{
    cout << "Khong co mat hang nao duoc ban.\n";
}

delete[] dsMaHang;
delete[] dsTongSoLuong;
}
```

B5: Chương trình kiểm tra biến `soLoaiHang`. Nếu lớn hơn 0, hệ thống khởi tạo giả định mặt hàng đầu tiên (index 0) trong danh sách thống kê là mặt hàng bán chạy nhất (`maHangBanChay` và `maxSoLuongBan`). Hàm sẽ sử dụng vòng lặp `for` chạy từ vị trí thứ 2 đến hết danh sách. Tại mỗi bước, hệ thống so sánh `dsTongSoLuong[i]` với giá trị `maxSoLuongBan` hiện tại. Nếu tìm thấy một mặt hàng có số lượng bán lớn hơn, hệ thống sẽ cập nhật lại giá trị cực đại và mã hàng tương ứng.

B6: Chương trình sẽ in ra kết quả mặt hàng bán chạy nhất cùng với số lượng đã bán, ngoài ra cũng sẽ dự báo nên nhập thêm hàng nào bán chạy.

B7: Giải phóng bộ nhớ động được phát sinh từ chương trình qua lệnh `delete[] dsMaHang` và `delete[] dsTongSoLuong`. Đây là bước cực kỳ quan trọng về mặt kỹ thuật. Việc này giúp trả lại vùng nhớ tạm đã cấp phát cho hệ điều hành, đảm bảo chương trình không bị chiếm dụng tài nguyên lãng phí sau khi rời khỏi hàm.

4.7. Tối ưu kho

```

void toiUuKho(const MatHang* kho, int nKho, const HoaDonXuat* dsHDX, int nHDX){
    if (nHDX == 0) {
        cout << "Chua co du lieu ban hang de toi uu kho.\n";
        return;
    }

    // Lấy danh sách sản phẩm duy nhất từ kho
    DoanhSoMatHang* dsDoanhSo = new DoanhSoMatHang[nKho];
    int soLoaiHang = 0;

    for (int i = 0; i < nKho; i++) {
        // Kiểm tra xem mã này đã có trong dsDoanhSo chưa
        bool tonTai = false;
        for (int j = 0; j < soLoaiHang; j++) {
            if (dsDoanhSo[j].maHang == kho[i].maHang) {
                tonTai = true; break;
            }
        }

        if (!tonTai) {
            dsDoanhSo[soLoaiHang].maHang = kho[i].maHang;
            dsDoanhSo[soLoaiHang].soLuongBan = 0;

            // Tính tổng số lượng đã bán cho sản phẩm này từ tất cả hoá đơn xuất
            for (int h = 0; h < nHDX; h++) {
                for (int k = 0; k < dsHDX[h].soLuongMatHang; k++) {
                    if (dsHDX[h].danhSachBan[k].maHang == kho[i].maHang) {
                        dsDoanhSo[soLoaiHang].soLuongBan += dsHDX[h].danhSachBan[k].soLuong;
                    }
                }
            }
            soLoaiHang++;
        }
    }
}

```

4.7.1. Khai báo hàm:

kho, nKho: Danh sách hàng hóa hiện có trong kho.

dsHDX, nHDX: Dữ liệu lịch sử xuất hàng để làm căn cứ phân tích.

Sử dụng hằng số const cho các tham số đầu vào để đảm bảo tính toàn vẹn của dữ liệu gốc, ngăn chặn các thay đổi ngoài ý muốn trong quá trình phân tích.

4.7.2. Các bước triển khai logic:

B1: Sử dụng mảng động dsDoanhSo (kiểu DoanhSoMatHang) để lưu trữ danh sách các mã hàng duy nhất và tổng lượng bán tương ứng. Chương trình duyệt qua danh sách kho, kiểm tra xem mã hàng đã tồn tại trong mảng thống kê chưa (biến tonTai). Nếu chưa, hệ thống sẽ tính tổng số lượng đã bán của mã hàng đó bằng cách duyệt qua toàn bộ chi tiết của tất cả hóa đơn xuất hàng (dsHDX).

```

    }

    //Sắp xếp ds theo số lượng bán giảm dần
    for (int i = 0; i < soLoaiHang - 1; i++) {
        for (int j = 0; j < soLoaiHang - i - 1; j++) {
            if (dsDoanhSo[j].soLuongBan < dsDoanhSo[j + 1].soLuongBan) {
                DoanhSoMatHang temp = dsDoanhSo[j];
                dsDoanhSo[j] = dsDoanhSo[j + 1];
                dsDoanhSo[j + 1] = temp;
            }
        }
    }

    // Hiển thị danh sách mặt hàng theo thứ tự doanh số
    //setw(n) căn chỉnh khoảng trắng sau n ký tự, left căn lề trái.
    cout << "\n=== Danh sách mặt hàng theo doanh số ===\n";
    cout << left << setw(15) << "Mã Hàng" << setw(15) << "Tổng Bán" << "Vị Trí De Xuất\n";
    for (int i = 0; i < soLoaiHang; i++) {
        string viTri;
        if (i < 3) viTri = "KHU VUC A (Gần cửa)";
        else if (i < 7) viTri = "KHU VUC B (Giữa kho)";
        else viTri = "KHU VUC C (Cuối kho)";

        cout << left << setw(15) << dsDoanhSo[i].maHang << setw(15) << dsDoanhSo[i].soLuongBan << viTri << "\n";
    }

    delete[] dsDoanhSo;
}

```

B2: Sử dụng thuật toán Bubble Sort (Sắp xếp nổi bọt) để sắp xếp mảng dsDoanhSo theo thứ tự số lượng bán giảm dần, với mục đích là đưa những mặt hàng có nhu cầu cao nhất lên vị trí đầu tiên trong danh sách xử lý.

B3: Chương trình chia kho hàng thành 3 khu vực chiến lược dựa trên thứ hạng doanh số: Khu vực A (Top 3 bán chạy nhất, gần cửa), Khu vực B (Từ hạng 4 đến hạng 7, giữa kho) và Khu vực C (Các mặt hàng còn lại, cuối kho).

B4: Chương trình sẽ sử dụng lệnh delete[] dsDoanhSo để giải phóng vùng nhớ tạm sau khi hoàn tất hiển thị báo cáo, tránh tình trạng chiếm dụng RAM không cần thiết.

4.8. Lưu dữ liệu quản lý (SaveData)

```

void SaveData(const MatHang* kho, int nKho, const HoaDonNhap* dsHDN, int nHDN, const HoaDonXuat* dsHDX, int nHDX){
    ofstream f("Kho_va_HoaDon.txt");
    if (!f.is_open()){
        cout << "Loi! Khong the tao/mo file.\n";
        return;
    }

    //Luu thông tin kho hàng
    f << nKho << "\n";
    for (int i = 0; i < nKho; i++){
        f << kho[i].maHang << "\n";
        f << kho[i].tenHang << "\n";
        f << kho[i].donViTinh << "\n";
        f << kho[i].giaNhap << " " << kho[i].giaBan << " " << kho[i].soLuongTon << "\n";
        f << kho[i].hanSuDung.ngay << " " << kho[i].hanSuDung.thang << " " << kho[i].hanSuDung.nam << "\n";
        f << kho[i].mucTonToiThieu << "\n";
    }

    //Luu thông tin hoá đơn nhập
    f << nHDN << "\n";
    for (int i = 0; i < nHDN; i++) {
        f << dsHDN[i].maHDN << "\n";
        f << dsHDN[i].ngayNhap.ngay << " " << dsHDN[i].ngayNhap.thang << " " << dsHDN[i].ngayNhap.nam << "\n";
        f << dsHDN[i].ncc.tenNCC << "\n"; // Theo Level 2 bạn mới nâng cấp
        f << dsHDN[i].ncc.soDienThoai << "\n";
        f << dsHDN[i].ncc.diaChi << "\n";
        f << dsHDN[i].ncc.diemChatLuong << "\n";
        f << dsHDN[i].ncc.diemUyTin << "\n";

        f << dsHDN[i].soLuongMatHang << "\n";
        for (int j = 0; j < dsHDN[i].soLuongMatHang; j++) {
            f << dsHDN[i].danhSachNhap[j].maHang << " " << dsHDN[i].danhSachNhap[j].soLuong << " " << dsHDN[i].danhSachNhap[j].donGia << "\n";
        }
        f << dsHDN[i].tongTien << "\n";
    }
}

```

4.8.1. Khai báo hàm:

kho, dsHDN, dsHDX: Các mảng dữ liệu chính của hệ thống.

Sử dụng hằng số const cho các tham số đầu vào để đảm bảo tính toàn vẹn của dữ liệu gốc, ngăn chặn các thay đổi ngoài ý muốn trong quá trình phân tích.

4.8.2. Các bước triển khai logic:

B1: Sử dụng ofstream f("Kho_va_HoaDon.txt") để mở tệp. Nếu tệp đã tồn tại, nội dung cũ sẽ bị ghi đè để cập nhật trạng thái mới nhất của kho hàng.

B2: Sử dụng !f.is_open() để kiểm tra lỗi. Nếu không thể tạo hoặc mở file (do lỗi quyền truy cập hoặc ổ đĩa đầy), hệ thống sẽ thông báo lỗi và dừng thao tác để bảo vệ tính toàn vẹn của logic.

B3: Lưu thông tin kho hàng bằng cách ghi số lượng mặt hàng nKho lên đầu phần dữ liệu kho để làm căn cứ cho việc đọc file sau này. Bằng cách dùng vòng lặp for để duyệt qua từng mặt hàng trong mảng, chương trình sẽ ghi lần lượt các thuộc tính như: Mã hàng, tên, đơn vị tính, các loại giá, số lượng tồn, hạn sử dụng và mức tồn tối thiểu.

B4: Lưu thông tin hoá đơn nhập thông qua biến nHDN. Với mỗi hóa đơn, ghi thông tin cơ bản và thông tin chi tiết về Nhà cung cấp (bao gồm cả các điểm đánh giá uy tín). Sau đó, sử dụng một vòng lặp phụ để ghi lại danh sách mặt hàng bên trong mỗi hóa đơn nhập.

```

//Lưu thông tin hoá đơn xuất
f << nHDX << "\n";
for (int i = 0; i < nHDX; i++){
    f << dsHDX[i].maHDX << "\n";
    f << dsHDX[i].ngayXuat.ngay << " " << dsHDX[i].ngayXuat.thang << " " << dsHDX[i].ngayXuat.nam << "\n";
    f << dsHDX[i].khachHang << "\n";

    f << dsHDX[i].soLuongMatHang << "\n";
    for (int j = 0; j < dsHDX[i].soLuongMatHang; j++){
        f << dsHDX[i].danhSachBan[j].maHang << " " << dsHDX[i].danhSachBan[j].soLuong << " " << dsHDX[i].danhSachBan[j].donGia << "\n";
    }
    f << dsHDX[i].tongTien << "\n";
}

f.close(); //Close the text file
cout << "Da ghi file thanh cong.\n";
}

```

B5: Tương tự, chương trình sẽ lưu thông tin hoá đơn xuất.

B6: Gọi lệnh `f.close()` để đóng tệp tin, đảm bảo dữ liệu được đẩy hoàn toàn từ bộ nhớ đệm vào đĩa cứng và giải phóng tài nguyên hệ thống.

4.9. Đọc dữ liệu quản lý (ReadData)

```

void ReadData(MatHang*& kho, int& nKho, int& sucChuaKho, HoaDonNhap*& dsHDN, int& nHDN, int& sucChuaHDN, HoaDonXuat*& dsHDX, int& nHDX, int& sucChuaHDX) {
    ifstream f ("Kho_va_HoaDon.txt");
    if(!f.is_open()){
        cout << "Loi! Khong the mo file.\n";
        return;
    }

    // Trước khi đọc dữ liệu mới, giải phóng bộ nhớ cũ nếu đã có
    // Dữ liệu được xóa theo thứ tự kho -> hoá đơn nhập -> hoá đơn xuất để tránh lỗi truy cập bộ nhớ
    if (kho != nullptr) {
        delete[] kho;
        kho = nullptr;
    }

    if (dsHDN != nullptr) {
        for (int i = 0; i < nHDN; i++) delete[] dsHDN[i].danhSachNhap;
        delete[] dsHDN;
        dsHDN = nullptr;
    }

    if (dsHDX != nullptr) {
        for (int i = 0; i < nHDX; i++) delete[] dsHDX[i].danhSachBan;
        delete[] dsHDX;
        dsHDX = nullptr;
    }
}

```

4.9.1. Khai báo hàm:

`*& kho, *& dsHDN, *& dsHDX`: Truyền tham chiếu đến con trỏ (Pointer Reference) để hàm có thể thay đổi trực tiếp địa chỉ vùng nhớ mà các con trỏ ở hàm main đang quản lý.

`& nKho, & nHDN, & nHDX`: Cập nhật số lượng phần tử thực tế.

`& sucChua...`: Cập nhật giới hạn tối đa của các mảng động sau khi nạp dữ liệu.

```

    case 9: {
        char confirm;
        bool valid = false;
        cout << "\n=== DOC DU LIEU TU FILE ===\n";
        cout << "CANH BAO: Thao tac nay se xoa sach cac thay doi CHUA LUU tren RAM.\n";

        // Vòng lặp cưỡng ép: Không cho thoát ra nếu không gõ đúng y hoặc n
        while (!valid) {
            cout << "Ban co chac chan muon tiep tuc? (y/n): ";
            cin >> confirm;
            confirm = tolower(confirm); // Chấp nhận cả Y và y, N và n

            if (confirm == 'y' || confirm == 'n') {
                valid = true; // Thoát vòng lặp
            } else {
                // Nếu gõ ký tự khác, thông báo và yêu cầu gõ lại
                cout << "Ky tu khong hop le! Chi chap nhan 'y' hoac 'n'.\n";
            }
        }

        if (confirm == 'y') {
            ReadData(kho, nKho, sucChuaKho, dsHDN, nHDN, sucChuaHDN, dsHDX, nHDX, sucChuaHDX);
        }
        else {
            cout << "Da huy thao tac. Du lieu hien tai van duoc giu nguyen.\n";
        }
        break;
    }
}

```

4.9.2. Các bước khai triển logic:

B1: Trước khi nạp dữ liệu mới, chương trình kiểm tra các con trỏ hiện tại. Nếu khác nullptr, hệ thống tiến hành giải phóng bộ nhớ cũ bằng delete[]. Chương trình sẽ hỏi xác nhận có xóa hàm cũ trong bộ nhớ RAM hiện tại không, nếu có, hàm ReadData mới được chạy.

```

f >> nKho;
sucChuaKho = nKho + 10;
kho = new MatHang[sucChuaKho];

for(int i = 0; i < nKho; i++){
    f >> kho[i].maHang;
    f.ignore();
    getline(f, kho[i].tenHang);
    getline(f, kho[i].donViTinh);
    f >> kho[i].giaNhap >> kho[i].giaBan >> kho[i].soLuongTon;
    f >> kho[i].hanSuDung.ngay >> kho[i].hanSuDung.thang >> kho[i].hanSuDung.nam;
    f >> kho[i].mucTonToiThieu;
}

f >> nHDN;
sucChuaHDN = nHDN + 10;
dsHDN = new HoaDonNhap[sucChuaHDN];

for(int i = 0; i < nHDN; i++){
    f >> dsHDN[i].maHDN;
    f >> dsHDN[i].ngayNhap.ngay >> dsHDN[i].ngayNhap.thang >> dsHDN[i].ngayNhap.nam;
    f.ignore();
    getline(f, dsHDN[i].ncc.tenNCC);
    getline(f, dsHDN[i].ncc.soDienThoai);
    getline(f, dsHDN[i].ncc.diaChi);
    f >> dsHDN[i].ncc.diemChatLuong;
    if (dsHDN[i].ncc.diemChatLuong < 1) dsHDN[i].ncc.diemChatLuong = 1;
    if (dsHDN[i].ncc.diemChatLuong > 5) dsHDN[i].ncc.diemChatLuong = 5;
    f >> dsHDN[i].ncc.diemUyTin;
    if (dsHDN[i].ncc.diemUyTin < 1) dsHDN[i].ncc.diemUyTin = 1;
    if (dsHDN[i].ncc.diemUyTin > 5) dsHDN[i].ncc.diemUyTin = 5;

    f >> dsHDN[i].soLuongMatHang;
    dsHDN[i].danhSachNhap = new ChiTietHoaDon[dsHDN[i].soLuongMatHang];
    for (int j = 0; j < dsHDN[i].soLuongMatHang; j++) {
        f >> dsHDN[i].danhSachNhap[j].maHang >> dsHDN[i].danhSachNhap[j].soLuong >> dsHDN[i].danhSachNhap[j].donGia;
    }
    f >> dsHDN[i].tongTien;
}

```

```

f >> nHDX;
sucChuaHDX = nHDX + 10;
dsHDX = new HoaDonXuat[sucChuaHDX];

for (int i = 0; i < nHDX; i++){
    f >> dsHDX[i].maHDX;
    f >> dsHDX[i].ngayXuat.ngay >> dsHDX[i].ngayXuat.thang >> dsHDX[i].ngayXuat.nam;
    f.ignore();
    getline(f, dsHDX[i].khachHang);

    f >> dsHDX[i].soLuongMatHang;
    dsHDX[i].danhSachBan = new ChiTietHoaDon[dsHDX[i].soLuongMatHang];
    for (int j = 0; j < dsHDX[i].soLuongMatHang; j++){
        f >> dsHDX[i].danhSachBan[j].maHang >> dsHDX[i].danhSachBan[j].soLuong >> dsHDX[i].danhSachBan[j].donGia;
    }
    f >> dsHDX[i].tongTien;
}

f.close();
cout << "Da doc file thanh cong!\n";
}

```

B2: Nạp dữ liệu Kho hàng bằng cách đọc số lượng mặt hàng nKho từ file. Sau đó thiết lập sucChuaKho = nKho + 10 (tạo khoảng trống dự phòng) và cấp phát vùng nhớ mới bằng lệnh new MatHang[...]. Ngoài ra còn sử dụng f.ignore() trước các hàm getline() để loại bỏ ký tự xuống dòng tồn đọng trong bộ đệm, đảm bảo đọc đúng tên hàng và đơn vị tính.

B3: Tương tự, sẽ nạp dữ liệu hoá đơn nhập và xuất. Tuy nhiên, khi đọc điểm diemChatLuong và diemUyTin của nhà cung cấp, chương trình kiểm tra nếu giá trị

nằm ngoài khoảng $[1, 5]$ thì tự động đưa về biên gần nhất (1 hoặc 5) để đảm bảo tính hợp lệ của dữ liệu phân tích. Sau đó, với mỗi hóa đơn, chương trình đọc `soLuongMatHang` và thực hiện cấp phát `new ChiTietHoaDon[...]` ngay lập tức để nạp chi tiết các món hàng.

B4: Đóng file và thông báo thành công.

5. Hàm main

5.1: Khởi tạo dữ liệu

```
//III. Main

int main(){
// 1.Khởi tạo kho
    MatHang* kho = nullptr;
    int nKho = 0;
    int sucChuaKho = 0;

// 2.Khởi tạo danh sách hoá đơn nhập
    HoaDonNhap* dsHDN = nullptr;
    int nHDN = 0;
    int sucChuaHDN = 0;

// 3.Khởi tạo danh sách hoá đơn xuất
    HoaDonXuat* dsHDX = nullptr;
    int nHDX = 0;
    int sucChuaHDX = 0;
```

- Khai báo các con trỏ và biến đếm để quản lý dữ liệu:

- + **kho, dsHDN, dsHDX**: Các con trỏ kiểu cấu trúc (MatHang*, HoaDonNhap*, HoaDonXuat*) dùng để quản lý mảng động cho hàng hóa và hóa đơn.
- + **nKho, nHDN, nHDX**: Các biến nguyên dùng để lưu trữ số lượng phần tử thực tế đang có trong danh sách.
- + **sucChuaKho, sucChuaHDN, sucChuaHDX**: Các biến nguyên quản lý kích thước

tối đa của mảng động tại một thời điểm (phục vụ việc cấp phát lại bộ nhớ).

+ **choice**: Biến số nguyên lưu trữ lựa chọn chức năng từ người dùng.

5.2: Menu lựa chọn

```
// Menu vòng lặp - người dùng chọn chức năng
int choice;
do{
    cout << "\n=====\\n";
    cout << "    UNG DUNG QUAN LY KHO SIEU THI\\n";
    cout << "=====\\n";
    cout << "1. Nhap hang vao kho\\n";
    cout << "2. Xuat hang ra kho\\n";
    cout << "3. Kiem tra mat hang sap het han\\n";
    cout << "4. Kiem tra mat hang sap het hang\\n";
    cout << "5. Thong ke ton kho\\n";
    cout << "6. Bao cao xu huong thi truong\\n";
    cout << "7. Toi uu kho\\n";
    cout << "8. Luu du lieu\\n";
    cout << "9. Doc du lieu\\n";
    cout << "0. Thoat\\n";
    cout << "=====\\n";
    cout << "Nhap lua chon: "; cin >> choice;
```

```

switch (choice){
    case 1: {
        cout << "\n===Nhap hang vao kho===\n";
        xuLyNhapHang(kho, nKho, sucChuaKho, dsHDN, nHDN, sucChuaHDN);
        break;
    }
    case 2: {
        cout << "\n===Xuat hang ra kho===\n";
        xuLyXuatHang(kho, nKho, sucChuaKho, dsHDX, nHDX, sucChuaHDX);
        break;
    }
    case 3: {
        cout << "\n===Kiem tra mat hang sap het han===\n";
        Ngay homNay;
        cout << "Nhap ngay hien tai (ngay -> thang -> nam): "; cin >> homNay.ngay >> homNay.thang >> homNay.nam; // lấy ngày cứng trước khi kiểm tra
        canhBaoHetHan(kho, nKho, homNay);
        break;
    }
    case 4: {
        cout << "\n===Kiem tra mat hang sap het hang===\n";
        canhBaoHetHang(kho, nKho);
        break;
    }
    case 5: {
        cout << "\n===Thong ke ton kho===\n";
        if (nKho == 0)
            cout << "Kho dang trong, khong co mat hang nao de thong ke.\n";
        else thongKeTonKho(kho, nKho);
        break;
    }
    case 6: {
        cout << "\n===Bao cao thong minh===\n";
        baoCaoThongMinh(kho, nKho, dsHDX, nHDX);
        break;
    }
    case 7: {
        cout << "\n===Toi uu kho===\n";
        toiUuKho(kho, nKho, dsHDX, nHDX);
        break;
    }
    case 8: {
        cout << "\n===Luu du lieu===\n";
        SaveData(kho, nKho, dsHDN, nHDN, dsHDX, nHDX);
        break;
    }
}

```

```

    case 9: {
        char confirm;
        bool valid = false;
        cout << "\n=== DOC DU LIEU TU FILE ===\n";
        cout << "CANH BAO: Thao tac nay se xoa sach cac thay doi CHUA LUU tren RAM.\n";

        // Vòng lặp cưỡng ép: Không cho thoát ra nếu không gõ đúng y hoặc n
        while (!valid) {
            cout << "Ban co chac chan muon tiep tuc? (y/n): ";
            cin >> confirm;
            confirm = tolower(confirm); // Chấp nhận cả Y và y, N và n

            if (confirm == 'y' || confirm == 'n') {
                valid = true; // Thoát vòng lặp
            } else {
                // Nếu gõ ký tự khác, thông báo và yêu cầu gõ lại
                cout << "Ky tu khong hop le! Chi chap nhan 'y' hoac 'n'.\n";
            }
        }

        if (confirm == 'y') {
            ReadData(kho, nKho, sucChuaKho, dsHDN, nHDN, sucChuaHDN, dsHDX, nHDX, sucChuaHDX);
        }
        else {
            cout << "Da huy thao tac. Du lieu hien tai van duoc giu nguyen.\n";
        }
        break;
    }

    case 0: {
        cout << "Dang thoat ung dung...\n";
        break;
    }
    default: {
        cout << "Lua chon khong hop le, vui long chon lai!\n";
        break;
    }
}
} while (choice != 0);

```

5.2.1. Cơ chế lặp duy trì giao diện (do-while)

- Toàn bộ Menu được đặt trong cấu trúc vòng lặp do-while. Điều này đảm bảo:

+Hiện thị ít nhất một lần: Menu sẽ luôn hiện ra ngay khi khởi động chương trình.

+Tính liên tục: Sau khi thực hiện xong một chức năng (ví dụ: nhập hàng xong), chương trình không kết thúc ngay mà tự động quay lại hiển thị Menu để người dùng thực hiện tác vụ tiếp theo.

+Điều kiện thoát: Vòng lặp chỉ dừng lại khi người dùng nhập đúng số 0 (while (choice != 0)).

5.2.2. Tiếp nhận và Phân luồng xử lý (switch-case)

- Biến **choice** đóng vai trò là "người điều hướng". Khi người dùng nhập một con số, cấu trúc **switch** sẽ so khớp và kích hoạt khối lệnh tương ứng:

+**Các Case 1 - 7:** Là các tính năng nghiệp vụ trực tiếp (Nhập, Xuất, Cảnh báo, Thống kê, Tối ưu). Chương trình gọi đến các hàm chuyên biệt để giữ cho mã nguồn trong hàm **main** gọn gàng.

+**Case 8 - 9:** Là các tính năng quản lý tệp tin. Điểm đáng chú ý là sự tách biệt giữa việc lưu trữ dữ liệu trên RAM và dữ liệu vật lý trên ổ cứng.

+**Default:** Đây là "chốt chặn an toàn", xử lý trường hợp người dùng nhập các số nằm ngoài danh sách (ví dụ: nhập số 11). Nó sẽ thông báo lỗi và yêu cầu chọn lại.

5.2.3. Cơ chế xác nhận bảo mật tại Case 9

- Khác với các chức năng khác, phần **Đọc dữ liệu** có thêm một lớp bảo vệ bằng vòng lặp **while (!valid)**.

+**Mục đích:** Ngăn chặn việc người dùng vô tình ghi đè dữ liệu cũ từ file lên dữ liệu mới đang xử lý trên RAM.

+**Logic xử lý:** Chương trình ép buộc người dùng phải trả lời bằng ký tự **y** (đồng ý) hoặc **n** (hủy). Mọi ký tự khác đều bị coi là không hợp lệ và người dùng không thể thoát khỏi bước này cho đến khi đưa ra quyết định rõ ràng.

5.2.4. Danh mục tính năng

- Menu được sắp xếp theo trình tự:

+**Đầu vào/Đầu ra:** Nhập hàng và Xuất hàng (1, 2).

+**Giám sát:** Kiểm tra hạn dùng và số lượng tồn (3, 4, 5).

+**Phân tích nâng cao:** Báo cáo xu hướng và tối ưu hóa diện tích kho (6, 7).

+**Lưu trữ:** Đảm bảo an toàn dữ liệu (8, 9).

5.3. Giải phóng bộ nhớ đã cấp phát động

```
// 4. Giải phóng bộ nhớ đã cấp phát động
delete[] kho;
for (int i = 0; i < nHDN; i++) delete[] dsHDN[i].danhsachNhap;
delete[] dsHDN;
for (int i = 0; i < nHDX; i++) delete[] dsHDX[i].danhsachBan;
delete[] dsHDX;
return 0;
}
```

- Khai báo các đối tượng cần xử lý:

+**kho**: Mảng động lưu trữ danh sách các mặt hàng hiện có.

+**dsHDN** / **dsHDX**: Các mảng động lưu trữ danh sách hóa đơn nhập và hóa đơn xuất.

+**danhsachNhap** / **danhsachBan**: Các mảng động "con" nằm bên trong mỗi cấu trúc (struct) hóa đơn.

+**nHDN** / **nHDX**: Số lượng hóa đơn thực tế đã được tạo ra để xác định số lần lặp khi dọn dẹp.

5.4: Các bước triển khai logic

B1: Khởi tạo tất cả các con trỏ dữ liệu về giá trị *nullptr* và các biến đếm, biến sức chứa về giá trị **0** để đảm bảo an toàn bộ nhớ ban đầu.

B2: Thiết lập vòng lặp *do-while* để duy trì ứng dụng, giúp menu luôn hiển thị lại sau khi mỗi chức năng kết thúc cho đến khi người dùng chọn thoát.

B3: Hiển thị giao diện menu trực quan với 9 chức năng chính và sử dụng `cin >> choice` để tiếp nhận yêu cầu từ người dùng.

B4: Sử dụng cấu trúc điều khiển *switch-case* để phân luồng xử lý:

-Gọi các hàm nghiệp vụ tương ứng (Nhập, Xuất, Cảnh báo, Thống kê...) dựa trên giá trị của *choice*.

-Truyền các tham số theo dạng tham chiếu.

B5: Tại chức năng đọc dữ liệu (Case 9), triển khai vòng lặp *while(!valid)* để cưỡng ép xác nhận *y/n*. Bước này giúp bảo vệ dữ liệu hiện tại trên RAM không bị ghi đè ngoài ý muốn.

B6: Khi biến *choice* nhận giá trị **0**, vòng lặp kết thúc. Chương trình thực hiện giải phóng bộ nhớ động theo quy trình ngược:

-Dùng vòng lặp *for* để xóa các mảng con nằm bên trong các cấu trúc hóa đơn (*danhSachNhap, danhSachBan*).

-Dùng lệnh *delete[]* để xóa các mảng chính (*kho, dsHDN, dsHDX*) nhằm trả lại tài nguyên cho hệ thống và tránh rò rỉ bộ nhớ.

6: Kết luận

6.1: ưu, nhược điểm khi dùng ứng dụng

***ưu điểm:**

-**Tối ưu hiệu suất và tốc độ:** Vì được viết bằng C++, ứng dụng có tốc độ thực thi rất nhanh và tiêu tốn ít tài nguyên hệ thống.

-**Quản lý dữ liệu linh hoạt:** Việc sử dụng **cấp phát bộ nhớ động** (*new, delete[]*) cho phép danh sách mặt hàng và hóa đơn co giãn theo nhu cầu thực tế, không bị giới hạn bởi các con số cố định ngay từ đầu.

-**Tính năng phân tích thông minh:** Ứng dụng tích hợp các hàm như *baoCaoThongMinh* và *toiUuKho*, giúp nhà quản lý không chỉ theo dõi số lượng mà còn dự báo được xu hướng thị trường và sắp xếp kho bãi khoa học.

-Độ tin cậy và an toàn dữ liệu:

+Cơ chế xác nhận (*y/n*) khi đọc dữ liệu giúp ngăn chặn việc mất dữ liệu ngoài ý muốn trên RAM.

+Phân tách rõ ràng giữa dữ liệu tạm thời (RAM) và dữ liệu lâu dài (File), giúp người dùng có quyền kiểm soát thời điểm lưu trữ chính xác.

-Tiết kiệm chi phí: Đây là dạng ứng dụng console, không yêu cầu phần cứng mạnh hay hệ quản trị cơ sở dữ liệu phức tạp, phù hợp cho các siêu thị quy mô vừa và nhỏ.

***nhược điểm:**

-Rủi ro về quản lý bộ nhớ: Do quản lý bằng con trỏ và cấp phát thủ công, nếu lập trình viên quên giải phóng bộ nhớ (như logic `delete[ ]` cuối hàm `main`), ứng dụng sẽ bị lỗi **Memory Leak** (rò rỉ bộ nhớ), khiến máy tính chạy chậm dần theo thời gian.

-Giao diện hạn chế: Sử dụng giao diện dòng lệnh (Console) có thể gây khó khăn cho những nhân viên không am hiểu về kỹ thuật. Việc thiếu hình ảnh và đồ họa khiến dữ liệu khó quan sát trực quan hơn các giao diện hiện đại (GUI).

-Hạn chế về môi trường đa người dùng: Mã nguồn hiện tại phù hợp cho việc quản lý đơn máy. Nếu muốn nhiều nhân viên cùng nhập/xuất kho một lúc, hệ thống sẽ gặp khó khăn trong việc đồng bộ hóa dữ liệu.

-Nhập liệu thủ công: Một số tính năng như kiểm tra hạn sử dụng (Case 3) vẫn yêu cầu người dùng nhập ngày hiện tại bằng tay, dễ dẫn đến sai sót do yếu tố con người.

-Cấu trúc dữ liệu tuyến tính: Việc tìm kiếm và thống kê trên mảng động (Array) có thể trở nên chậm hơn khi danh sách hàng hóa lên đến hàng chục nghìn mặt hàng.

Kết luận: Ứng dụng này là một giải pháp **gọn nhẹ, mạnh mẽ và hiệu quả** cho việc quản lý kho nội bộ. Tuy nhiên, nó đòi hỏi người quản lý phải có quy trình lưu trữ dữ liệu kỷ luật và lập trình viên phải cực kỳ cẩn thận trong việc kiểm soát bộ nhớ động.